



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе № 5
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Обучение на основе глубоких Q-сетей"

Выполнил:

Студент группы ИУ5Ц-21М
Папин А.В.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Постановка задачи.....	4
Теоретические сведения	5
Алгоритм DQN	5
Архитектуры нейронных сетей.....	6
QNetwork (базовый DQN).....	6
DuelingQNetwork	6
QNetworkBN.....	7
QNetworkLSTM	7
Теоретическое обоснование	8
Результаты экспериментов.....	9
Анализ результатов	10
QNetwork (базовый DQN).....	10
DuelingQNetwork	10
QNetworkBN.....	10
QNetworkLSTM	10
Ключевые наблюдения	10
Демонстрация	12
Вывод.....	15
Исходный код программ.....	16
Главная программа (main.py).....	17
Класс DQNAgent (agent.py)	23
Архитектуры сетей (network.py).....	31
Буфер опыта (replay_buffer.py).....	36
Визуализация агента (demo.py).....	38
Управление запуска (Makefile)	42

Цель работы

Целью лабораторной работы является ознакомление с базовыми методами обучения с подкреплением на основе глубоких Q-сетей.

Задание

1. На основе рассмотренных на лекции примеров реализуйте алгоритм DQN.
2. В качестве среды можно использовать классические среды (в этом случае используется полносвязная архитектура нейронной сети).
3. В качестве среды можно использовать игры Atari (в этом случае используется сверточная архитектура нейронной сети).
4. В случае реализации среды на основе сверточной архитектуры нейронной сети +1 балл за экзамен.

Постановка задачи

Среда: CartPole-v1 (Gymnasium)

Задача: Удерживать шест на тележке как можно дольше

Параметры среды:

- State space: 4 (позиция тележки, скорость, угол, угловая скорость)
- Action space: 2 (влево, вправо)
- Максимальная награда: 500

Параметры обучения:

Параметр	Значение
Количество эпизодов	50 / 100 / 500
Размер скрытого слоя	128
Размер батча	32
Learning rate	0.003
Gamma (discount)	0.99
Epsilon decay	0.98
Buffer capacity	10 000
Target update	50 шагов

Теоретические сведения

Алгоритм DQN

Deep Q-Network (DQN) – это алгоритм обучения с подкреплением, который использует глубокую нейронную сеть для аппроксимации Q-функции.

Основные компоненты:

- Q-Network – НС, аппроксимирующая Q-значения $Q(s, a)$
- Target Network – копия Q-network, используемая для стабилизации обучения
- Experience Replay – буфер для хранения переходов (s, a, r, s')
- Epsilon-greedy – стратегия исследования

Алгоритм:

1. Инициализировать Q-network и target network
2. Заполнить буфер опыта случайными действиями
3. For each episode:
 - 3.1. Получить состояние s
 - 3.2. Выбрать действие a (epsilon-greedy)
 - 3.3. Выполнить действие, получить (r, s')
 - 3.4. Сохранить переход в буфер
 - 3.5. Если достаточно данных в буфере:
 - 3.5.1. Случайная выборка батча
 - 3.5.2. Вычислить target: $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$
 - 3.5.3. Обновить Q-network (MSE loss)
 - 3.6. Каждые target_update шагов: обновить target network

В данной работе реализованы и исследованы 4 архитектуры:

- QNetwork – базовая архитектура с 3 полносвязными слоями
- DuelingQNetwork – разделяет оценку $V(s)$ и преимущества $A(s,a)$
- QNetworkBN – с BatchNorm и Dropout для регуляризации
- QNetworkLSTM – с LSTM слоем для запоминания истории

Архитектуры нейронных сетей

QNetwork (базовый DQN)

Трёхслойная полносвязная сеть для аппроксимации Q-функции:

$$Q(s, a; \theta) = f_3(f_2(f_1(s)))$$

где:

- $s \in \mathbb{R}^4$ – состояние (CartPole)
- $a \in \{0,1\}$ – действие
- $f_i(x) = \text{ReLU}(W_i x + b_i)$ – линейный слой с ReLU активацией

Архитектура:

- Input: 4 → 128 (fc1)
- Hidden: 128 → 128 (fc2)
- Output: 128 → 2 (fc3)

Число параметров:

$$4 \cdot 128 + 128 + 128 \cdot 128 + 128 + 128 \cdot 2 + 2 = 17\,410$$

DuelingQNetwork

Разделяет Q-функцию на Value и Advantage:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$$

где:

- $V(s)$ – ценность состояния (скаляр)
- $A(s, a)$ – преимущество действия (вектор)

Архитектура:

- Feature: 4 → 128 → 128 (общие признаки)
- Value: 128 → 64 → 1 ($V(s)$)
- Advantage: 128 → 64 → 2 ($A(s, a)$)

Число параметров: 33 859

QNetworkBN

Добавляет BatchNorm для стабилизации:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$
$$y = \gamma \hat{x} + \beta$$

где:

- μ, σ – статистики батча,
- γ, β – обучаемые параметры.

Архитектура:

$$s \xrightarrow{fc1} BN \xrightarrow{ReLU} Dropout \xrightarrow{fc2} BN \xrightarrow{ReLU} Dropout \xrightarrow{fc3} Q(s,a)$$

Число параметров: 17 922

QNetworkLSTM

Использует LSTM для запоминания последовательностей:

$$h_t, c_t = LSTM(x_t, h_{t-1}, c_{t-1})$$
$$Q(s, a) = W_o \cdot h_T + b_o$$

где h_T – последнее скрытое состояние LSTM.

Архитектура:

- LSTM: $4 \rightarrow 128$ (однонаправленный, 1 слой)
- FC: $128 \rightarrow 2$

Число параметров: 68 866

Теоретическое обоснование

QNetwork

Простая архитектура с тремя полносвязными слоями. ReLU активация добавляет нелинейность, достаточную для аппроксимации Q-функции в простых задачах типа CartPole.

Dueling DQN

Идея основана на декомпозиции Q-функции (Wang et al., 2016):

- $V(s)$ оценивает "насколько хорошо находится в состоянии s "
- $A(s,a)$ оценивает "насколько действие a лучше других"

Разделение позволяет лучше оценивать состояния без действий, что полезно для задач с многими действиями.

BatchNorm

Batch Normalization (Ioffe & Szegedy, 2015) нормализует входы каждого слоя:

- Стабилизирует градиенты
- Ускоряет сходимость
- Регуляризует модель

Однако в RL показывает нестабильность из-за нестационарности данных.

LSTM

Добавляет память для запоминания последовательностей:

- Обучается на последовательностях состояний
- Может улавливать темпоральные зависимости
- Полезно для частично наблюдаемых сред (POMDP)

Для CartPole избыточно, так как задача Марковская (полное наблюдение).

Результаты экспериментов

Таблица 1 - Результаты (50 эпизодов)

Архитектура	Время (сек)	Средняя награда	Максимум	Минимум	Параметры
QNetwork	62.7	500.0	500.0	500.0	17,410
DuelingQNetwork	77.4	126.4	133.0	120.0	33,859
QNetworkBN	62.8	403.7	426.0	381.0	17,922
QNetworkLSTM	81.7	224.8	236.0	215.0	68,866

Таблица 2 - Результаты (100 эпизодов)

Архитектура	Время (сек)	Средняя награда	Максимум	Минимум	Параметры
QNetwork	118.8	500.0	500.0	500.0	17,410
DuelingQNetwork	149.2	176.8	193.0	161.0	33,859
QNetworkBN	121.3	20.1	31.0	10.0	17,922
QNetworkLSTM	156.8	235.5	252.0	219.0	68,866

Таблица 3 - Результаты (500 эпизодов)

Архитектура	Время (сек)	Средняя награда	Максимум	Минимум	Параметры
QNetworkLSTM	843.3	154.7	158.0	149.0	68,866
DuelingQNetwork	607.2	116.3	120.0	111.0	33,859
QNetwork	492.9	112.3	116.0	106.0	17,410
QNetworkBN	162.1	74.6	80.0	68.0	17,922

Анализ результатов

QNetwork (базовый DQN)

- 50 эпох: 500.0 (максимум!)
- 100 эпох: 500.0 (максимум!)
- 500 эпох: 112.3 (упал!)

Вывод: Overfitting при долгом обучении. Лучший результат для простых задач.

DuelingQNetwork

- 50 эпох: 126.4
- 100 эпох: 176.8
- 500 эпох: 116.3

Вывод: Нестабилен, не дает преимуществ для простых задач.

QNetworkBN

- 50 эпох: 403.7
- 100 эпох: 20.1
- 500 эпох: 74.6

Вывод: BatchNorm нестабилен в RL задачах.

QNetworkLSTM

- 50 эпох: 224.8
- 100 эпох: 235.5
- 500 эпох: 154.7

Вывод: Устойчивее к долгому обучению, но избыточен для CartPole.

Ключевые наблюдения

- 100 эпизодов – оптимально для QNetwork, дальше – переобучение
- LSTM устойчивее к долгому обучению
- BatchNorm показывает худшую стабильность в RL задачах

- Dueling не дает преимуществ для простых задач

Таблица 4 - Сравнение архитектуры

Архитектура	Параметры	Формула
QNetwork	17 410	$Q = f_3(f_2(f_1(s)))$
DuelingQNetwork	33 859	$Q = V + A - \bar{A}$
QNetworkBN	17 922	$Q = f_3(BN(f_2(BN(f_1(s)))))$
QNetworkLSTM	68 866	$Q = W_o \cdot LSTM(s)$

Демонстрация

```
redalex@redalex-Nitro-AN515-44:~/GitHub/ML_AS0IU$ make
LR5: DQN (Deep Q-Network) - Обучение с подкреплением

Основные команды:
make install      - Установить зависимости
make train        - Обучить агента (базовый DQN, 50 эпизодов)
make mlflow       - Запустить MLflow UI
make mlflow-migrate - Мигрировать БД MLflow
make demo         - Запустить визуализацию
make test         - Быстрый тест агента
make clean        - Очистить временные файлы

Архитектуры нейросетей:
make train-qnetwork - Базовая DQN (3 слоя, ReLU)
make train-dueling  - Dueling DQN (разделение V и A)
make train-bn       - DQN с BatchNorm и Dropout
make train-lstm     - DQN с LSTM (память)
make train-all      - Обучить все 4 модели для сравнения

Параметры обучения:
EPOCHS, BATCH_SIZE, LR, GAMMA, EPSILON_DECAY, EPSILON_MIN
HIDDEN_DIM, TARGET_UPDATE, BUFFER_CAPACITY, WARMUP_STEPS, NETWORK
```

Рисунок 1 - Демонстрация работы makefile

```
redalex@redalex-Nitro-AN515-44:~/GitHub/ML_AS0IU$ make train EPOCHS=50
python3 main.py --network qnetwork --episodes 50 --batch-size 32 --lr 0.003 --gamma 0.99 --hid
den-dim 128 --warmup-steps 1000
2026/05/11 15:27:41 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/05/11 15:27:41 INFO mlflow.store.db.utils: Updating database tables
2026/05/11 15:27:42 INFO mlflow.tracking.fluent: Experiment with name 'DQN_CartPole_Comparison
' does not exist. Creating a new experiment.
=====
MLflow включен: d59c70ec8d91483cbd509c6066bd832c
=====
Используется устройство: cuda
Состояние: (4,), Действий: 2
Warmup: заполнение буфера...
Буфер: 277 переходов

Архитектура модели:
=====
Layer (type:depth-idx)      Output Shape      Param #
=====
QNetwork
├─Linear: 1-1                [1, 128]          640
├─Linear: 1-2                [1, 128]          16,512
├─Linear: 1-3                [1, 2]            258
=====
Total params: 17,410
Trainable params: 17,410
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.02
=====
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.07
```

Рисунок 2 - Демонстрация обучения модели

```

Total mult-adds (Units.MEGABYTES): 0.02
=====
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.07
Estimated Total Size (MB): 0.07
=====

Информация о модели:
  Устройство: cuda
  GPU: NVIDIA GeForce GTX 1650 Ti
  Архитектура: QNetwork(4 -> 128 -> 2)
  Буфер: 10000

Параметры: episodes=50, batch=32, lr=0.003
Начало обучения: 50 эпизодов, warmup=1000 шаров
Episode 10/50 | Reward: 33.0 | Steps: 33 | Epsilon: 0.010
Episode 20/50 | Reward: 221.0 | Steps: 221 | Epsilon: 0.010
Episode 30/50 | Reward: 250.0 | Steps: 250 | Epsilon: 0.010
Episode 40/50 | Reward: 199.0 | Steps: 199 | Epsilon: 0.010
Episode 50/50 | Reward: 183.0 | Steps: 183 | Epsilon: 0.010

Время обучения: 35.0 сек
2026/05/11 15:28:20 WARNING mlflow.models.model: 'artifact_path' is deprecated. Please use 'name' instead.
2026/05/11 15:28:20 WARNING mlflow.pytorch: Saving pytorch model by Pickle or CloudPickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is to set 'export_model' to True to save the pytorch model using the safe graph model format.
2026/05/11 15:28:20 WARNING mlflow.utils.requirements_utils: Found torch version (2.10.0+cu128) contains a local version label (+cu128). MLflow logged a pip requirement for this package as 'torch==2.10.0' without the local version label to make it installable from PyPI. To specify pip requirements containing local version labels, please use 'conda_env' or 'pip_requirements'.
2026/05/11 15:28:27 WARNING mlflow.utils.requirements_utils: Found torch version (2.10.0+cu128) contains a local version label (+cu128). MLflow logged a pip requirement for this package as 'torch==2.10.0' without the local version label to make it installable from PyPI. To specify pip requirements containing local version labels, please use 'conda_env' or 'pip_requirements'.

Оценка агента...
Средняя награда: 201.2
Максимум: 228.0
Минимум: 182.0

MLflow логирование завершено

Готово!

```

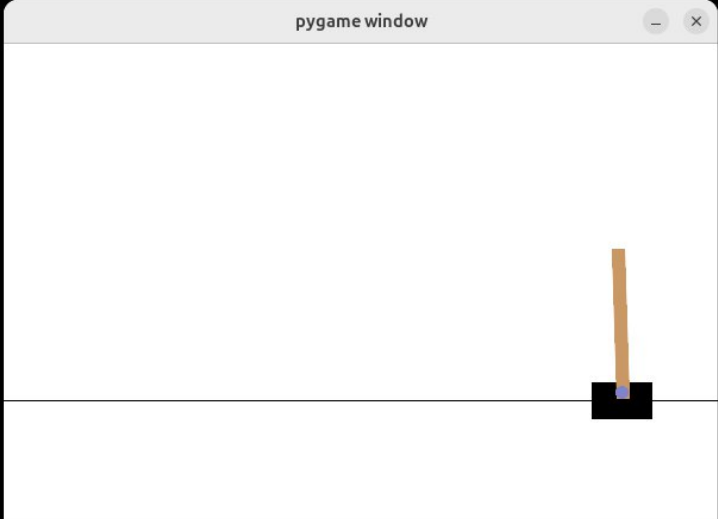
Рисунок 3 - Демонстрация обучения модели

```

redalexddad@redalexddad-Nitro-AN515-44:~/GitHub/ML_AS0IU$ make demo MODEL=/home/redalexddad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb843339fbf6dc354826016/artifacts
python3 demo.py --model /home/redalexddad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb843339fbf6dc354826016/artifacts
Загрузка модели: /home/redalexddad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb843339fbf6dc354826016/artifacts
Тип сети: qnetwork
  Автоопределение типа сети: lstm
  Загрузка через MLflow...
  MLflow path: /home/redalexddad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb843339fbf6dc354826016/artifacts
2026/05/11 15:01:00 WARNING mlflow.pytorch: Stored model version '2.9.1+cu128' does not match installed version '2.10.0+cu128'.
Создание окружения с визуализацией...

Демонстрация игры (3 эпизодов):
Эпизод 1/3

```



```

redalexdad@redalexdad-Nitro-AN515-44:~/GitHub/ML_AS0IU$ make demo MODEL=/home/r
edalexdad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb843339fbf6dc354826016/ar
tifacts/data/model.pth
python3 demo.py --model /home/redalexdad/GitHub/ML_AS0IU/mlruns/1/models/m-b23a
ec5f5fb843339fbf6dc354826016/artifacts/data/model.pth --episodes 3 --delay 0.02
Загрузка модели: /home/redalexdad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb
843339fbf6dc354826016/artifacts/data/model.pth
Тип сети: qnetwork
Автоопределение типа сети: lstm
Загрузка через MLflow...
MLflow path: /home/redalexdad/GitHub/ML_AS0IU/mlruns/1/models/m-b23aec5f5fb84
3339fbf6dc354826016/artifacts
2026/05/11 15:01:00 WARNING mlflow.pytorch: Stored model version '2.9.1+cu128'
does not match installed PyTorch version '2.10.0+cu128'
Создание окружения с визуализацией...

Демонстрация игры (3 эпизодов):

Эпизод 1/3
Награда: 153.0, Шагов: 153

Эпизод 2/3
Награда: 154.0, Шагов: 154

Эпизод 3/3
Награда: 158.0, Шагов: 158

Готово!
redalexdad@redalexdad-Nitro-AN515-44:~/GitHub/ML_AS0IU$ █

```

Рисунок 4 - Демонстрация работы загрузки обученной модели qnetwork по типу сети LSTM

Вывод

В ходе лабораторной работы были исследованы 4 архитектуры DQN для задачи CartPole-v1:

1. Базовая QNetwork показала лучшие результаты (500.0 при 50-100 эпизодах) благодаря простоте и отсутствию переобучения на коротких тренировках.
2. DuelingQNetwork не дал преимуществ для простой задачи CartPole, где нет необходимости разделять оценку состояния и действия.
3. QNetworkBN продемонстрировал нестабильность – BatchNorm плохо работает с RL из-за зависимости от распределения входных данных.
4. QNetworkLSTM оказался устойчивее при долгом обучении (500 эпизодов), но для CartPole избыточен – задача не требует памяти о прошлых состояниях.

Исходный код программ

Основные файлы проекта:

1. `main.py` – главная программа обучения
2. `dqn/agent.py` – класс DQNAgent
3. `dqn/network.py` – архитектуры сетей
4. `dqn/replay_buffer.py` – буфер опыта
5. `demo.py` – визуализация агента
6. `Makefile` – управление запуска

Главная программа (main.py)

```
"""
LR5: DQN (Deep Q-Network) - Обучение с подкреплением
Задача: CartPole-v1 (балансировка шеста)

Вариант 2
Студент: Папин А.В., ИУ5Ц-21М
"""

import numpy as np
import torch
import gymnasium as gym
import argparse
import os
import datetime

from dqn import DQNAgent
from src.visualization import plot_results, plot_detailed_analysis
from src.train import train as train_agent
from src.evaluate import evaluate as evaluate_agent
try:
    import mlflow
    import mlflow.pytorch # type: ignore
    MLFLOW_AVAILABLE = True
except ImportError:
    MLFLOW_AVAILABLE = False

def log_model_info(agent: DQNAgent, env: gym.Env, device: torch.device,
args) -> dict:
    from torchinfo import summary

    model_summary = summary(agent.q_network, input_size=(1, 4),
verbose=False)

    info = {
        "environment": {
            "name": "CartPole-v1",
            "observation_space": str(env.observation_space),
            "action_space": str(env.action_space),
        },
        "model": {
            "state_dim": 4,
            "action_dim": agent.action_dim,
            "hidden_dim": args.hidden_dim,
            "total_parameters": model_summary.total_params,
            "trainable_parameters": model_summary.trainable_params,
        },
        "hyperparameters": {
            "lr": args.lr,
            "gamma": args.gamma,
```

```

        "epsilon": 1.0,
        "epsilon_decay": args.epsilon_decay,
        "epsilon_min": args.epsilon_min,
        "buffer_capacity": args.buffer_capacity,
    },
    "system": {
        "device": str(device),
        "cuda_available": torch.cuda.is_available(),
        "cuda_device_name": torch.cuda.get_device_name(0) if
torch.cuda.is_available() else None,
        "torch_version": torch.__version__,
    },
}
return info

def parse_args():
    parser = argparse.ArgumentParser(description='DQN для CartPole-v1')
    parser.add_argument('--episodes', type=int, default=50,
help='Количество эпизодов')
    parser.add_argument('--batch-size', type=int, default=32, help='Размер
батча')
    parser.add_argument('--hidden-dim', type=int, default=128,
help='Размер скрытого слоя')
    parser.add_argument('--network', type=str, default='qnetwork',
choices=['qnetwork', 'dueling', 'bn', 'lstm'],
help='Тип сети: qnetwork (базовая), dueling
(Dueling DQN), bn (с BatchNorm)')
    parser.add_argument('--lr', type=float, default=0.003, help='Скорость
обучения')
    parser.add_argument('--gamma', type=float, default=0.99,
help='Коэффициент дисконтирования')
    parser.add_argument('--epsilon-decay', type=float, default=0.98,
help='Затухание epsilon')
    parser.add_argument('--epsilon-min', type=float, default=0.01,
help='Минимальный epsilon')
    parser.add_argument('--target-update', type=int, default=50,
help='Частота обновления целевой сети')
    parser.add_argument('--buffer-capacity', type=int, default=10000,
help='Размер буфера')
    parser.add_argument('--seed', type=int, default=42, help='Seed для
воспроизводимости')
    parser.add_argument('--warmup-steps', type=int, default=1000,
help='Шаги warmup')
    parser.add_argument('--eval-episodes', type=int, default=10,
help='Эпизоды для оценки')
    parser.add_argument('--model-path', type=str, default='dqn_model.pth',
help='Путь для сохранения модели')
    parser.add_argument('--mlflow', action='store_true', default=True,
help='Включить MLflow логгирование')
    parser.add_argument('--no-mlflow', action='store_true',
help='Отключить MLflow логгирование')

```

```

    return parser.parse_args()

def log_params(args):
    mlflow.log_params({
        'episodes': args.episodes,
        'batch_size': args.batch_size,
        'hidden_dim': args.hidden_dim,
        'lr': args.lr,
        'gamma': args.gamma,
        'epsilon_decay': args.epsilon_decay,
        'epsilon_min': args.epsilon_min,
        'target_update': args.target_update,
        'buffer_capacity': args.buffer_capacity,
        'seed': args.seed,
        'warmup_steps': args.warmup_steps,
    })

def log_metrics(rewards, losses, q_values, results):
    mlflow.log_metric('train_mean_reward', np.mean(rewards))
    mlflow.log_metric('train_max_reward', max(rewards))
    mlflow.log_metric('train_min_reward', min(rewards))

    if losses:
        mlflow.log_metric('train_mean_loss', np.mean(losses))

    if q_values:
        mlflow.log_metric('train_mean_q', np.mean(q_values))

    mlflow.log_metric('eval_mean_reward', results['mean_reward'])
    mlflow.log_metric('eval_max_reward', results['max_reward'])
    mlflow.log_metric('eval_min_reward', results['min_reward'])
    mlflow.log_metric('eval_std_reward', results['std_reward'])

def main():
    args = parse_args()

    np.random.seed(args.seed)
    torch.manual_seed(args.seed)

    use_mlflow = args.mlflow and not args.no_mlflow and MLFLOW_AVAILABLE

    if use_mlflow:
        mlflow.set_experiment('DQN_CartPole_Comparison')
        active = mlflow.active_run()
        if active is None:
            mlflow.start_run(run_name=f"{args.network}_{datetime.datetime.now().strftime('%Y%m%d_%H%M')}")
        else:
            mlflow.start_run(run_id=active.info.run_id)
            log_params(args)

```

```

print(f"{' '*50}")
run = mlflow.active_run()
print(f"MLflow включен: {run.info.run_id if run else 'N/A'}")
print(f"{' '*50}")

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используется устройство: {device}")

if use_mlflow:
    mlflow.log_param('device', str(device))
    mlflow.log_param('network', args.network)

env = gym.make('CartPole-v1')
obs_space = env.observation_space
action_space = env.action_space
obs_shape = obs_space.shape
action_dim = int(action_space.n) # type: ignore
print(f"Состояние: {obs_shape}, Действий: {action_dim}")

agent = DQNAgent(
    state_dim=int(obs_shape[0]), # type: ignore
    action_dim=action_dim,
    hidden_dim=args.hidden_dim,
    network_type=args.network,
    lr=args.lr,
    gamma=args.gamma,
    epsilon_decay=args.epsilon_decay,
    epsilon_min=args.epsilon_min,
    target_update=args.target_update,
    buffer_capacity=args.buffer_capacity,
    device=device
)

print("Warmup: заполнение буфера...")
for _ in range(10):
    state, _ = env.reset()
    for _ in range(100):
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, _ =
env.step(action)
        done = terminated or truncated
        agent.store_transition(state, action, float(reward),
next_state, done)
        state = next_state
        if done:
            break
print(f"Буфер: {len(agent.buffer)} переходов")

if use_mlflow:
    from torchinfo import summary

```

```

print("\nАрхитектура модели:")
summary(agent.q_network, input_size=(1, 4),
col_names=["output_size", "num_params"], verbose=True)

model_summary = summary(agent.q_network, input_size=(1, 4),
verbose=False)

import json
model_info = {
    "environment": {"name": "CartPole-v1"},
    "model": {
        "state_dim": 4,
        "action_dim": agent.action_dim,
        "hidden_dim": args.hidden_dim,
        "total_parameters": model_summary.total_params,
        "trainable_parameters": model_summary.trainable_params,
    },
    "hyperparameters": {
        "lr": args.lr,
        "gamma": args.gamma,
        "epsilon_decay": args.epsilon_decay,
        "epsilon_min": args.epsilon_min,
        "buffer_capacity": args.buffer_capacity,
    },
    "system": {
        "device": str(device),
        "cuda_available": torch.cuda.is_available(),
        "cuda_device_name": torch.cuda.get_device_name(0) if
torch.cuda.is_available() else None,
        "torch_version": torch.__version__,
    },
}

mlflow.log_dict(model_info, 'model_info.json')

print("\nИнформация о модели:")
print(f" Устройство: {model_info['system']['device']}")
if model_info['system']['cuda_device_name']:
    print(f" GPU: {model_info['system']['cuda_device_name']}")
print(f" Архитектура: QNetwork({model_info['model']['state_dim']}
-> {model_info['model']['hidden_dim']} ->
{model_info['model']['action_dim']})")
print(f" Буфер:
{model_info['hyperparameters']['buffer_capacity']}")

print(f"\nПараметры: episodes={args.episodes},
batch={args.batch_size}, lr={args.lr}")

start_time = datetime.datetime.now()

rewards, losses, q_values = train_agent(

```

```

        env=env,
        agent=agent,
        episodes=args.episodes,
        batch_size=args.batch_size,
        seed=args.seed,
        warmup_steps=args.warmup_steps,
        mlflow_logging=use_mlflow
    )

    training_time = (datetime.datetime.now() - start_time).total_seconds()
    print(f"\nВремя обучения: {training_time:.1f} сек")

    if use_mlflow:
        mlflow.log_metric('training_time', training_time)
        mlflow.pytorch.log_model(agent.q_network,
artifact_path=f'{args.network}_q_network')

    temp_plots = '/tmp/mlflow_plots'
    plot_results(rewards, losses, q_values, save_path=temp_plots)
    plot_detailed_analysis(rewards, losses, q_values,
save_path=temp_plots)

    if use_mlflow:
        mlflow.log_artifact(f'{temp_plots}/training_analysis.png')
        mlflow.log_artifact(f'{temp_plots}/training_progress.png')
        mlflow.log_artifact(f'{temp_plots}/detailed_analysis.png')

    import shutil
    shutil.rmtree(temp_plots, ignore_errors=True)

    print("\nОценка агента...")
    results = evaluate_agent(env, agent, episodes=args.eval_episodes)
    print(f"Средняя награда: {results['mean_reward']:.1f}")
    print(f"Максимум: {results['max_reward']}")
    print(f"Минимум: {results['min_reward']}")

    if use_mlflow:
        log_metrics(rewards, losses, q_values, results)
        mlflow.end_run()
        print("\nMLflow логирование завершено")

    env.close()
    print("\nГотово!")

if __name__ == '__main__':
    main()

```

Класс DQNAgent (agent.py)

```
"""
Модуль DQN агента.
Реализует основной алгоритм Deep Q-Network для обучения с подкреплением.
"""

import torch
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
from typing import Optional, Dict, Any

import mlflow # type: ignore

from dqn.network import QNetwork, DuelingQNetwork, QNetworkBN,
QNetworkLSTM
from dqn.buffer import ReplayBuffer

class DQNAgent:
    """
    Агент, реализующий алгоритм DQN (Deep Q-Network).

    Основные компоненты:
    1. Q-Network - нейросеть для оценки Q-значений  $Q(s, a)$ 
    2. Target Network - копия Q-сети для вычисления целевых значений
    3. Replay Buffer - буфер для хранения опыта
    4. Epsilon-greedy - стратегия исследования/эксплуатации

    Алгоритм обучения:
    1. Выбрать действие через epsilon-greedy политику
    2. Выполнить действие, получить  $(s, a, r, s', done)$ 
    3. Сохранить переход в буфер
    4. Если буфер достаточно заполнен - обучать на случайном батче:
        -  $Q(s, a)$  - текущая оценка
        -  $target = r + \gamma * \max_{a'} Q_{target}(s', a')$  - целевое значение
        (Bellman equation)
        - Минимизировать MSE между Q и target
    5. Периодически обновлять Target Network
    """

    def __init__(
        self,
        state_dim: int,
        action_dim: int,
        hidden_dim: int = 128,
        network_type: str = "qnetwork",
        lr: float = 0.001,
        gamma: float = 0.99,
        epsilon: float = 1.0,
        epsilon_decay: float = 0.995,
```

```

epsilon_min: float = 0.01,
target_update: int = 100,
buffer_capacity: int = 10000,
device: Optional[torch.device] = None,
experiment_name: str = "DQN_CartPole"
) -> None:
    """
    Инициализация DQN агента.

    Args:
        state_dim: Размерность пространства состояний
        action_dim: Количество возможных действий
        hidden_dim: Количество нейронов в скрытых слоях
        network_type: Тип сети ('qnetwork', 'dueling', 'bn')
        lr: Скорость обучения (learning rate)
        gamma: Коэффициент дисконтирования награды (0.95-0.99)
        epsilon: Начальная вероятность случайного действия
        epsilon_decay: Коэффициент уменьшения epsilon после каждого
шага
        epsilon_min: Минимальное значение epsilon (не прекращаем
исследование полностью)
        target_update: Частота обновления целевой сети (каждые N
шагов)
        buffer_capacity: Размер буфера опыта
        device: Устройство для вычислений (CPU/CUDA)
        experiment_name: Имя эксперимента для MLflow
    """
    # Выбор архитектуры сети
    network_classes = {
        'qnetwork': QNetwork,
        'dueling': DuelingQNetwork,
        'bn': QNetworkBN,
        'lstm': QNetworkLSTM
    }
    NetworkClass = network_classes.get(network_type, QNetwork)

    # Параметры действий и награды
    self.action_dim = action_dim
    self.gamma = gamma #  $\gamma$  - коэффициент дисконтирования
    self.network_type = network_type

    # Параметры epsilon-greedy стратегии
    self.epsilon = epsilon # Текущее значение epsilon
    self.epsilon_decay = epsilon_decay # Во сколько раз уменьшаем
epsilon
    self.epsilon_min = epsilon_min # Минимальное значение epsilon

    self.target_update = target_update # Интервал обновления target
сети
    self.steps = 0 # Счетчик шагов обучения

```



```

self.experiment_name = experiment_name

# Определение устройства (CPU или GPU)
if device is None:
    device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')
self.device = device

# Инициализация сетей
# Q-Network - обучаемая сеть (выбранного типа)
self.q_network = NetworkClass(state_dim, action_dim,
hidden_dim).to(self.device)

# Target Network - фиксированная цель для стабильности
# Инициализируется весами Q-network
self.target_network = NetworkClass(state_dim, action_dim,
hidden_dim).to(self.device)
self.target_network.load_state_dict(self.q_network.state_dict())
self.target_network.eval() # Режим инференса (без обучения)

# Оптимизатор - Adam с настраиваемым lr
self.optimizer = optim.Adam(self.q_network.parameters(), lr=lr)

# Буфер опыта для Experience Replay
self.buffer = ReplayBuffer(buffer_capacity)

# Логирование гиперпараметров в MLflow
self._log_hyperparameters(locals())

def _log_hyperparameters(self, locals_dict: Dict[str, Any]) -> None:
    """Логирование гиперпараметров в MLflow."""
    hp = {
        "hidden_dim": locals_dict.get("hidden_dim"),
        "lr": locals_dict.get("lr"),
        "gamma": locals_dict.get("gamma"),
        "epsilon_decay": locals_dict.get("epsilon_decay"),
        "epsilon_min": locals_dict.get("epsilon_min"),
        "target_update": locals_dict.get("target_update"),
        "buffer_capacity": locals_dict.get("buffer_capacity"),
        "state_dim": locals_dict.get("state_dim"),
        "action_dim": locals_dict.get("action_dim"),
    }
    mlflow.log_params(hp)

def select_action(self, state: np.ndarray, training: bool = True) ->
int:
    """
    Выбор действия на основе текущей политики.

    Epsilon-greedy стратегия:
    - С вероятностью epsilon: случайное действие (исследование)

```

- С вероятностью $1-\epsilon$: действие с максимальным Q-значением (эксплуатация)

Args:

state: Текущее состояние

training: Флаг режима обучения (влияет на epsilon-greedy)

Returns:

Выбранное действие (целое число)

"""

Исследование: случайное действие

if training and np.random.random() < self.epsilon:

return np.random.randint(self.action_dim)

Эксплуатация: выбираем действие с максимальным Q-значением

self.q_network.eval()

with torch.no_grad():

state_tensor =

torch.FloatTensor(state).unsqueeze(0).to(self.device)

q_values = self.q_network(state_tensor)

action = q_values.argmax().item()

self.q_network.train()

return action

def store_transition(

self,

state: np.ndarray,

action: int,

reward: float,

next_state: np.ndarray,

done: bool

) -> None:

"""

Сохранение перехода в буфер опыта.

Args:

state: Текущее состояние s

action: Выбранное действие a

reward: Полученная награда r

next_state: Следующее состояние s'

done: Флаг завершения эпизода

"""

self.buffer.push(state, action, reward, next_state, done)

def train_step(self, batch_size: int) -> Optional[float]:

"""

Один шаг обучения на случайном батче из буфера.

Алгоритм (Double DQN с MSE loss):

1. sampling: Выбрать случайный батч из буфера

2. Q-values: $Q(s, a)$ - текущие оценки для выбранных действий

3. Target: $r + \gamma * \max_a Q_{\text{target}}(s', a')$ - целевое значение
 - Используем target_network для устойчивости
 - (1 - done) обнуляет награду для terminal states
4. Loss: MSE(current_q, target_q)
5. Backprop: обновить веса Q-network
6. Update: обновить target network и epsilon

Args:

batch_size: Размер батча для обучения

Returns:

Значение функции потерь или None если буфер слишком мал

"""

Не обучаем если недостаточно данных в буфере

```
if len(self.buffer) < batch_size:
    return None
```

1. Получить случайный батч из буфера

states, actions, rewards, next_states, dones =

```
self.buffer.sample(batch_size)
```

2. Конвертировать в тензоры PyTorch

```
states = torch.FloatTensor(states).to(self.device)
```

```
actions = torch.LongTensor(actions).to(self.device)
```

```
rewards = torch.FloatTensor(rewards).to(self.device)
```

```
next_states = torch.FloatTensor(next_states).to(self.device)
```

```
dones = torch.FloatTensor(dones).to(self.device)
```

3. Вычислить текущие Q-значения для выбранных действий

gather(1, actions.unsqueeze(1)) выбирает Q(s, a) для конкретных действий

```
current_q = self.q_network(states).gather(1,
actions.unsqueeze(1)).squeeze(1)
```

4. Вычислить целевые Q-значения через Target Network

Double DQN: используем Q-network для выбора, target_network для оценки

```
with torch.no_grad():
```

```
    next_q = self.target_network(next_states).max(1)[0]
```

```
    # Уравнение Беллмана:  $Q(s, a) = r + \gamma * \max Q(s', a')$ 
```

```
    # Для terminal states (done=True) последний член обнуляется
```

```
    target_q = rewards + (1 - dones) * self.gamma * next_q
```

5. Вычислить и минимизировать потери (MSE Loss)

```
loss = F.mse_loss(current_q, target_q)
```

6. Обратное распространение

```
self.optimizer.zero_grad() # Обнулить градиенты
```

```
loss.backward() # Вычислить градиенты
```

Gradient clipping - предотвратить взрыв градиентов

```

        torch.nn.utils.clip_grad_norm_(self.q_network.parameters(),
max_norm=1.0)

        self.optimizer.step() # Обновить веса

        # 7. Обновить счетчик шагов
        self.steps += 1

        # 8. Периодически обновлять Target Network (политика фиксированных
целей)
        if self.steps % self.target_update == 0:

self.target_network.load_state_dict(self.q_network.state_dict())

        # 9. Уменьшать epsilon (меньше исследований со временем)
        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay

        return loss.item()

def log_metrics(
    self,
    episode: int,
    reward: float,
    loss: Optional[float],
    epsilon: float,
    q_values_mean: Optional[float] = None
) -> None:
    """
    Логирование метрик обучения в MLflow.

    Args:
        episode: Номер эпизода
        reward: Награда за эпизод
        loss: Значение функции потерь
        epsilon: Текущее значение epsilon
        q_values_mean: Среднее Q-значение
    """
    metrics = {
        "episode_reward": reward,
        "epsilon": epsilon,
    }
    if loss is not None:
        metrics["loss"] = loss
    if q_values_mean is not None:
        metrics["q_values_mean"] = q_values_mean

    mlflow.log_metrics(metrics, step=episode)

def log_model(self, artifact_path: str = "model") -> None:
    """

```

Сохранить модель в MLflow.

Args:

artifact_path: Путь для сохранения артефакта
"""

```
mlflow.pytorch.log_model( # type: ignore
    self.q_network,
    artifact_path,
    registered_model_name=f"{self.experiment_name}_model"
)
```

```
def save(self, path: str) -> None:
    """
```

Сохранить веса модели и состояние агента.

Сохраняемые компоненты:

- Веса Q-network
- Веса Target Network
- Состояние оптимизатора
- Текущий epsilon (политика)
- Количество шагов

Args:

path: Путь для сохранения файла
"""

```
# Перенести на CPU для сериализации
q_network_cpu = self.q_network.cpu().state_dict()
target_network_cpu = self.target_network.cpu().state_dict()
optimizer_cpu = self.optimizer.state_dict()
```

```
# Вернуть обратно на устройство
self.q_network.to(self.device)
self.target_network.to(self.device)
```

```
# Сохранить словарь состояния
torch.save({
    'q_network': q_network_cpu,
    'target_network': target_network_cpu,
    'optimizer': optimizer_cpu,
    'epsilon': self.epsilon,
    'steps': self.steps,
}, path)
```

```
# Логировать в MLflow
mlflow.log_artifact(path)
```

```
def load(self, path: str) -> None:
    """
```

Загрузить веса модели и состояние агента.

Args:

```
        path: Путь к файлу с сохраненными весами
    """
    checkpoint = torch.load(path, weights_only=False,
map_location=self.device)
    self.q_network.load_state_dict(checkpoint['q_network'])
    self.target_network.load_state_dict(checkpoint['target_network'])
    self.optimizer.load_state_dict(checkpoint['optimizer'])
    self.epsilon = checkpoint['epsilon']
    self.steps = checkpoint['steps']
```

Архитектуры сетей (network.py)

```
"""
```

Модуль нейронной сети для DQN агента.

Реализует аппроксимацию Q-функции: $Q(s, a) \rightarrow$ оценка качества действия в состоянии.

Доступные архитектуры:

1. QNetwork - базовая DQN сеть (3 слоя, ReLU)
2. DuelingQNetwork - Dueling DQN с разделением $V(s)$ и $A(s,a)$
3. QNetworkBN - DQN с BatchNorm для стабилизации

```
"""
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import mlflow
import mlflow.pytorch
from datetime import datetime
```

```
class QNetwork(nn.Module):
```

```
    """
```

Нейронная сеть для оценки Q-значений (Value Network).

Архитектура: 3 полносвязных слоя с ReLU активацией.

Вход: состояние окружения (state_dim,)

Выход: Q-значения для каждого действия (action_dim,)

Пример для CartPole-v1:

- state_dim = 4 (позиция, скорость, угол, угловая скорость)
- action_dim = 2 (влево или вправо)

```
    """
```

```
    def __init__(self, state_dim: int, action_dim: int, hidden_dim: int =
128) -> None:
```

```
        super(QNetwork, self).__init__()
```

```
        # Первый полносвязный слой: state -> hidden
```

```
        # Преобразует вектор состояния в скрытое представление
```

```
        self.fc1 = nn.Linear(state_dim, hidden_dim)
```

```
        # Второй полносвязный слой: hidden -> hidden
```

```
        # Дополнительная нелинейность для сложных зависимостей
```

```
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
```

```
        # Выходной слой: hidden -> action
```

```
        # Генерирует Q-значения для каждого возможного действия
```

```
        self.fc3 = nn.Linear(hidden_dim, action_dim)
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```

"""
Прямой проход через сеть.

Args:
    x: Тензор состояния формы (batch_size, state_dim)

Returns:
    Тензор Q-значений формы (batch_size, action_dim)
"""
# ReLU: f(x) = max(0, x) - добавляет нелинейность
x = F.relu(self.fc1(x))
x = F.relu(self.fc2(x))
return self.fc3(x)

class DuelingQNetwork(nn.Module):
    """
    Архитектура Dueling DQN - разделяет оценку состояния и преимущества
    действий.

    Основная идея Dueling DQN:
    
$$Q(s, a) = V(s) + A(s, a) - \text{avg}(A(s, a))$$


    где:
    - V(s) - ценность состояния (Value)
    - A(s, a) - преимущество действия (Advantage)

    Преимущества:
    - Лучше оценивает состояния без действий
    - Сходится быстрее для задач с многими действиями
    - Меньше параметров чем полная Q-сеть

    Вдохновлено: "Dueling Network Architectures for Deep Reinforcement
    Learning" (Wang et al., 2016)
    """

    def __init__(self, state_dim: int, action_dim: int, hidden_dim: int =
128) -> None:
        super(DuelingQNetwork, self).__init__()

        # Общие признаки - извлекают представление состояния
        self.feature = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )

        # Value stream - оценивает ценность состояния V(s)
        # Отвечает на вопрос: "Насколько хорошо находиться в этом
состоянии?"
        self.value_stream = nn.Sequential(

```



```

        nn.Linear(hidden_dim, hidden_dim // 2),
        nn.ReLU(),
        nn.Linear(hidden_dim // 2, 1) # Один выход - V(s)
    )

    # Advantage stream - оценивает преимущество каждого действия
A(s,a)
    # Отвечает на вопрос: "Насколько лучше это действие относительно
    других?"
    self.advantage_stream = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim // 2),
        nn.ReLU(),
        nn.Linear(hidden_dim // 2, action_dim) # action_dim выходов -
A(s,a)
    )

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Прямой проход через сеть.

    Args:
        x: Тензор состояния формы (batch_size, state_dim)

    Returns:
        Тензор Q-значений формы (batch_size, action_dim)
    """
    # Извлечение признаков
    features = self.feature(x)

    # Вычисление V(s) и A(s,a)
    value = self.value_stream(features) # (batch_size, 1)
    advantage = self.advantage_stream(features) # (batch_size,
action_dim)

    # Комбинация:  $Q(s,a) = V(s) + A(s,a) - \text{mean}(A(s,:))$ 
    # Вычитание среднего делает сеть идентифицируемой
    q_values = value + advantage - advantage.mean(dim=1, keepdim=True)

    return q_values

```

```

class QNetworkBN(nn.Module):
    """

```

QNetwork с BatchNorm для стабилизации обучения.

BatchNorm преимущества:

- Стабилизация градиентов (внутреннее смещение covariate shift)
- Ускорение сходимости
- Регуляризация (менее подвержен переобучению)
- Позволяет использовать более высокий learning rate

Вдохновлено: "Batch Normalization: Accelerating Deep Network Training

by Reducing Internal Covariate Shift" (Ioffe & Szegedy, 2015)

```
"""
def __init__(self, state_dim: int, action_dim: int, hidden_dim: int =
128) -> None:
    super(QNetworkBN, self).__init__()

    # BatchNorm нормализует входные данные каждого слоя
    self.bn1 = nn.BatchNorm1d(hidden_dim)
    self.bn2 = nn.BatchNorm1d(hidden_dim)

    self.fc1 = nn.Linear(state_dim, hidden_dim)
    self.fc2 = nn.Linear(hidden_dim, hidden_dim)
    self.fc3 = nn.Linear(hidden_dim, action_dim)

    # Dropout для дополнительной регуляризации
    # Случайно обнуляет нейроны во время обучения
    self.dropout = nn.Dropout(0.1)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Прямой проход через сеть с BatchNorm и Dropout.

    Args:
        x: Тензор состояния формы (batch_size, state_dim)

    Returns:
        Тензор Q-значений формы (batch_size, action_dim)
    """
    x = self.fc1(x)
    x = self.bn1(x)
    x = F.relu(x)
    x = self.dropout(x)

    x = self.fc2(x)
    x = self.bn2(x)
    x = F.relu(x)
    x = self.dropout(x)

    return self.fc3(x)
```

```
class QNetworkLSTM(nn.Module):
```

```
    """
```

QNetwork с LSTM для запоминания последовательностей.

LSTM преимущества:

- Запоминает временные зависимости между состояниями
- Лучше для частично наблюдаемых сред (POMDP)
- Может учить паттерны в последовательностях

Вход: последовательность состояний (seq_len, state_dim)

Выход: Q-значения для каждого действия
"""

```
def __init__(self, state_dim: int, action_dim: int, hidden_dim: int =
128, num_layers: int = 1) -> None:
    super(QNetworkLSTM, self).__init__()

    self.hidden_dim = hidden_dim
    self.num_layers = num_layers

    self.lstm = nn.LSTM(
        input_size=state_dim,
        hidden_size=hidden_dim,
        num_layers=num_layers,
        batch_first=True
    )

    self.fc = nn.Linear(hidden_dim, action_dim)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Прямой проход через сеть.

    Args:
        x: Тензор формы (batch_size, seq_len, state_dim)

    Returns:
        Тензор Q-значений формы (batch_size, action_dim)
    """
    if x.dim() == 2:
        x = x.unsqueeze(1)

    lstm_out, (h_n, c_n) = self.lstm(x)

    last_output = lstm_out[:, -1, :]

    return self.fc(last_output)
```

Буфер опыта (replay_buffer.py)

```
"""
```

Модуль буфера повторения опыта (Replay Buffer).

Используется для стабилизации обучения DQN - разрывает корреляцию между последовательными переходами.

```
"""
```

```
import numpy as np
import random
from collections import deque
from typing import Tuple, List
```

```
class ReplayBuffer:
```

```
    """
```

Буфер для хранения переходов (s, a, r, s', done).

Принцип работы:

1. Агент взаимодействует с окружением и сохраняет переходы в буфер
2. При обучении случайно выбираем батч из буфера (Experience Replay)
3. Это устраняет корреляцию между последовательными наблюдениями

Преимущества:

- Повторное использование опыта (эффективнее данных)
- Случайная выборка стабилизирует градиенты
- Разрывает временную зависимость между переходами

```
"""
```

```
def __init__(self, capacity: int = 10000) -> None:
```

```
    """
```

Инициализация буфера.

Args:

capacity: Максимальное количество хранимых переходов.

При достижении лимита старые переходы удаляются.

```
    """
```

deque - эффективная структура с O(1) добавлением/удалением

maxlen=capacity автоматически удаляет старые элементы

```
self.buffer: deque = deque(maxlen=capacity)
```

```
def push(self, state: np.ndarray, action: int, reward: float,
        next_state: np.ndarray, done: bool) -> None:
```

```
    """
```

Добавить переход в буфер.

Args:

state: Текущее состояние s

action: Выбранное действие a

reward: Полученная награда r

next_state: Следующее состояние s'

done: Флаг завершения эпизода (terminal state)

```

"""
self.buffer.append((state, action, reward, next_state, done))

def sample(self, batch_size: int) -> Tuple[np.ndarray, ...]:
    """
    Случайно выбрать батч переходов для обучения.

    Args:
        batch_size: Размер батча

    Returns:
        Кортеж массивов: (states, actions, rewards, next_states,
dones)
    """
    # Случайная выборка без повторений
    batch: List = random.sample(self.buffer, batch_size)

    # Распаковка батча на отдельные компоненты
    states, actions, rewards, next_states, dones = zip(*batch)

    return (
        np.array(states, dtype=np.float32),          # (batch_size,
state_dim)
        np.array(actions, dtype=np.int64),          # (batch_size,)
        np.array(rewards, dtype=np.float32),        # (batch_size,)
        np.array(next_states, dtype=np.float32),    # (batch_size,
state_dim)
        np.array(dones, dtype=np.float32)           # (batch_size,) -
1.0 если done, иначе 0.0
    )

def __len__(self) -> int:
    """Возвращает количество переходов в буфере."""
    return len(self.buffer)

```

Визуализация агента (demo.py)

```
"""
Демонстрация обученного агента (с визуализацией)
"""

import gymnasium as gym
from dqn import DQNAgent
from dqn.network import QNetwork, DuelingQNetwork, QNetworkBN,
QNetworkLSTM
import argparse
import time
import torch
import mlflow

def parse_args():
    parser = argparse.ArgumentParser(description='DQN Demo - визуализация
агента')
    parser.add_argument('--model', type=str, default='dqn_model.pth',
help='Путь к модели')
    parser.add_argument('--network', type=str, default='qnetwork',
choices=['qnetwork', 'dueling', 'bn', 'lstm'],
help='Тип архитектуры сети')
    parser.add_argument('--episodes', type=int, default=3,
help='Количество эпизодов')
    parser.add_argument('--delay', type=float, default=0.02,
help='Задержка между шагами (сек)')
    parser.add_argument('--hidden-dim', type=int, default=128,
help='Размер скрытого слоя')
    return parser.parse_args()

def create_network(network_type: str, state_dim: int, action_dim: int,
hidden_dim: int):
    """Создать сеть нужного типа."""
    if network_type == 'lstm':
        return QNetworkLSTM(state_dim, action_dim, hidden_dim)
    elif network_type == 'dueling':
        return DuelingQNetwork(state_dim, action_dim, hidden_dim)
    elif network_type == 'bn':
        return QNetworkBN(state_dim, action_dim, hidden_dim)
    else:
        return QNetwork(state_dim, action_dim, hidden_dim)

def detect_network_type(path: str) -> str:
    """Определить тип сети по структуре весов."""
    import os
    import torch

    model_path = path
    if 'mlruns' in path or 'mlflow' in path.lower():
        if path.endswith('model.pth'):
```

```

        model_path = path
    else:
        model_path = os.path.join(path, 'data', 'model.pth')

    if os.path.exists(model_path):
        state_dict = torch.load(model_path, map_location='cpu',
weights_only=False)

        if isinstance(state_dict, dict):
            if 'q_network' in state_dict:
                state_dict = state_dict['q_network']
            elif 'model' in state_dict:
                state_dict = state_dict['model']
        elif hasattr(state_dict, 'state_dict'):
            state_dict = state_dict.state_dict()

    keys = list(state_dict.keys()) if hasattr(state_dict, 'keys') else
[]

    if any('lstm' in k for k in keys):
        return 'lstm'
    elif any('fc_val' in k or 'fc_adv' in k for k in keys):
        return 'dueling'
    elif any('bn' in k or 'batch_norm' in k for k in keys):
        return 'bn'

    return 'qnetwork'

def load_model_for_demo(path: str, hidden_dim: int, network_type: str) ->
DQNAgent:
    """Загрузить модель - поддерживает как локальные файлы, так и
MLflow."""
    state_dim, action_dim = 4, 2

    if network_type == 'qnetwork' and ('mlruns' in path or 'mlflow' in
path.lower()):
        detected_type = detect_network_type(path)
        if detected_type != 'qnetwork':
            print(f" Автоопределение типа сети: {detected_type}")
            network_type = detected_type

    q_network = create_network(network_type, state_dim, action_dim,
hidden_dim)
    target_network = create_network(network_type, state_dim, action_dim,
hidden_dim)

    agent = DQNAgent(state_dim=state_dim, action_dim=action_dim,
hidden_dim=hidden_dim)
    agent.q_network = q_network
    agent.target_network = target_network

```

```

optimizer = torch.optim.Adam(agent.q_network.parameters(), lr=0.003)
agent.optimizer = optimizer

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
agent.device = device

if 'mlruns' in path or 'mlflow' in path.lower():
    print("  Загрузка через MLflow...")
    mlflow_path = path
    if path.endswith('model.pth'):
        mlflow_path = path.replace('/artifacts/data/model.pth',
'/artifacts')
    print(f"  MLflow path: {mlflow_path}")
    model = mlflow.pytorch.load_model(mlflow_path,
map_location=device)
    model.to(device)
    model.eval()

    if isinstance(model, torch.nn.Module):
        try:
            agent.q_network.load_state_dict(model.state_dict())
            agent.target_network.load_state_dict(model.state_dict())
            agent.q_network.to(device)
            agent.target_network.to(device)
        except RuntimeError as e:
            detected_type = detect_network_type(path)
            if detected_type != network_type:
                print(f"  Автоопределение типа сети: {detected_type}")
                network_type = detected_type
                q_network = create_network(network_type, state_dim,
action_dim, hidden_dim)
                target_network = create_network(network_type,
state_dim, action_dim, hidden_dim)
                agent.q_network = q_network
                agent.target_network = target_network
                agent.q_network.to(device)
                agent.target_network.to(device)
                agent.q_network.load_state_dict(model.state_dict())
            agent.target_network.load_state_dict(model.state_dict())
            agent.q_network.to(device)
            agent.target_network.to(device)
        else:
            raise
    else:
        agent.load(path)

    return agent

def play(agent, env, episodes=3, delay=0.02):
    agent.q_network.eval()

```



```

for episode in range(epochs):
    state, _ = env.reset()
    total_reward = 0
    steps = 0

    print(f"\nЭпизод {episode + 1}/{epochs}")

    while True:
        action = agent.select_action(state, training=False)
        next_state, reward, terminated, truncated, _ =
env.step(action)
        done = terminated or truncated

        total_reward += reward
        steps += 1
        state = next_state

        env.render()
        time.sleep(delay)

        if done:
            break

    print(f" Награда: {total_reward}, Шагов: {steps}")
    if total_reward >= 500:
        print("    МАКСИМАЛЬНАЯ НАГРАДА!")

env.close()
print("\nГотово!")

if __name__ == '__main__':
    args = parse_args()

    print(f"Загрузка модели: {args.model}")
    print(f"Тип сети: {args.network}")
    agent = load_model_for_demo(args.model, args.hidden_dim, args.network)

    print("Создание окружения с визуализацией...")
    env = gym.make('CartPole-v1', render_mode='human')

    print(f"\nДемонстрация игры ({args.epochs} эпизодов):")
    play(agent, env, episodes=args.epochs, delay=args.delay)

```

Управление запуска (Makefile)

.PHONY: help install train train-qnetwork train-dueling train-bn train-all
demo test clean mlflow mlflow-migrate

```
GREEN := $(shell tput setaf 2)
YELLOW := $(shell tput setaf 3)
BLUE := $(shell tput setaf 4)
RESET := $(shell tput sgr0)
```

help:

```
@echo "$(BLUE)LR5: DQN (Deep Q-Network)$(RESET) - Обучение с
подкреплением"
@echo ""
@echo "$(GREEN)Основные команды:$(RESET)"
@echo "  $(YELLOW)make install$(RESET)                - Установить
зависимости"
@echo "  $(YELLOW)make train$(RESET)                - Обучить агента
(базовый DQN, 50 эпизодов)"
@echo "  $(YELLOW)make mlflow$(RESET)                - Запустить
MLflow UI"
@echo "  $(YELLOW)make mlflow-migrate$(RESET)        - Мигрировать БД
MLflow"
@echo "  $(YELLOW)make demo$(RESET)                - Запустить
визуализацию"
@echo "  $(YELLOW)make test$(RESET)                - Быстрый тест
агента"
@echo "  $(YELLOW)make clean$(RESET)                - Очистить
временные файлы"
@echo ""
@echo "$(GREEN)Архитектуры нейросетей:$(RESET)"
@echo "  $(YELLOW)make train-qnetwork$(RESET)        - Базовая DQN (3
слоя, ReLU)"
@echo "  $(YELLOW)make train-dueling$(RESET)        - Dueling DQN
(разделение V и A)"
@echo "  $(YELLOW)make train-bn$(RESET)                - DQN с
BatchNorm и Dropout"
@echo "  $(YELLOW)make train-lstm$(RESET)                - DQN с LSTM
(память)"
@echo "  $(YELLOW)make train-all$(RESET)                - Обучить все 4
модели для сравнения"
@echo ""
@echo "$(GREEN)Параметры обучения:$(RESET)"
@echo "  EPISODES, BATCH_SIZE, LR, GAMMA, EPSILON_DECAY, EPSILON_MIN"
@echo "  HIDDEN_DIM, TARGET_UPDATE, BUFFER_CAPACITY, WARMUP_STEPS,
NETWORK"
```

install:

```
pip install -r requirements.txt
```

train:

```
python3 main.py --network $(NETWORK) --episodes $(EPISODES) --batch-size $(BATCH_SIZE) --lr $(LR) --gamma $(GAMMA) --hidden-dim $(HIDDEN_DIM) --warmup-steps $(WARMUP_STEPS) $(if $(NO_MLFLOW),--no-mlflow,)
```

train-qnetwork:

```
@echo "$(GREEN)Обучение базовой DQN сети...$(RESET)"
python3 main.py --network qnetwork --episodes $(EPISODES) --batch-size $(BATCH_SIZE) --lr $(LR) --gamma $(GAMMA) --hidden-dim $(HIDDEN_DIM)
```

train-dueling:

```
@echo "$(GREEN)Обучение Dueling DQN сети...$(RESET)"
python3 main.py --network dueling --episodes $(EPISODES) --batch-size $(BATCH_SIZE) --lr $(LR) --gamma $(GAMMA) --hidden-dim $(HIDDEN_DIM)
```

train-bn:

```
@echo "$(GREEN)Обучение DQN с BatchNorm...$(RESET)"
python3 main.py --network bn --episodes $(EPISODES) --batch-size $(BATCH_SIZE) --lr $(LR) --gamma $(GAMMA) --hidden-dim $(HIDDEN_DIM)
```

train-lstm:

```
@echo "$(GREEN)Обучение DQN с LSTM...$(RESET)"
python3 main.py --network lstm --episodes $(EPISODES) --batch-size $(BATCH_SIZE) --lr $(LR) --gamma $(GAMMA) --hidden-dim $(HIDDEN_DIM)
```

train-all:

```
@echo "$(GREEN)Обучение всех архитектур для сравнения...$(RESET)"
@echo ""
@echo "$(YELLOW)=== 1. QNetwork (базовый DQN) ===$(RESET)"
python3 main.py --network qnetwork --episodes $(EPISODES) --batch-size 32 --lr 0.003 --gamma 0.99 --hidden-dim 128
@echo ""
@echo "$(YELLOW)=== 2. DuelingQNetwork ===$(RESET)"
python3 main.py --network dueling --episodes $(EPISODES) --batch-size 32 --lr 0.003 --gamma 0.99 --hidden-dim 128
@echo ""
@echo "$(YELLOW)=== 3. QNetwork с BatchNorm ===$(RESET)"
python3 main.py --network bn --episodes $(EPISODES) --batch-size 32 --lr 0.003 --gamma 0.99 --hidden-dim 128
@echo ""
@echo "$(YELLOW)=== 4. QNetwork с LSTM ===$(RESET)"
python3 main.py --network lstm --episodes $(EPISODES) --batch-size 32 --lr 0.003 --gamma 0.99 --hidden-dim 128
@echo ""
@echo "$(GREEN)Обучение всех моделей завершено!$(RESET)"
```

demo:

```
python3 demo.py --model $(MODEL) --episodes $(DEMO_EPISODES) --delay $(DELAY)
```

test:

```
@python3 -c "from dqn import DQNAgent; agent = DQNAgent(state_dim=4,
```

```
action_dim=2); agent.load('${MODEL}'); print('${GREEN}Модель загружена
OK$(RESET)')
```

```
mlflow-migrate:
```

```
@echo "${GREEN}Миграция базы данных MLflow...$(RESET)"
```

```
mlflow db upgrade sqlite:///mlflow.db
```

```
mlflow:
```

```
@echo "${GREEN}Запуск MLflow UI...$(RESET)"
```

```
@echo "${YELLOW}Откройте:
```

```
http://127.0.0.1:5000/#/experiments/1/runs?columns=metrics.episode_reward,
metrics.episode_loss,metrics.episode_q,metrics.epsilon$(RESET)"
```

```
mlflow ui --host 127.0.0.1 --port 5000
```

```
clean:
```

```
rm -rf plots/*.png dqn_model.pth __pycache__ dqn/__pycache__
```

```
src/__pycache__ .pytest_cache
```

```
@echo "${GREEN}Очистка завершена$(RESET)"
```

```
EPISODES ?= 50
```

```
BATCH_SIZE ?= 32
```

```
LR ?= 0.003
```

```
GAMMA ?= 0.99
```

```
EPSILON_DECAY ?= 0.98
```

```
EPSILON_MIN ?= 0.01
```

```
HIDDEN_DIM ?= 128
```

```
TARGET_UPDATE ?= 50
```

```
BUFFER_CAPACITY ?= 10000
```

```
WARMUP_STEPS ?= 1000
```

```
NETWORK ?= qnetwork
```

```
MODEL ?= dqn_model.pth
```

```
DELAY ?= 0.02
```

```
DEMO_EPISODES ?= 3
```